Reproducing DAIN: Deep Adaptive Input Normalization for Limit Order Book Price Forecasting on FI-2010

Challa Umesh Varun (Roll No. 230318)
EE798R Pattern Recognition, Course Project

October 2026

Abstract

Limit order book data are non-stationary: price levels differ between stocks and drift during the day, which makes fixed input normalisation a weak point of deep models for high-frequency price forecasting. Deep Adaptive Input Normalization (DAIN) [1] is a trainable layer that shifts, scales and gates every input window according to its own statistics. This report re-implements DAIN in PyTorch and reproduces the ablation study of the paper on the FI-2010 benchmark with the authors' data: an MLP predicts the next mid-price direction from 15 order-book snapshots, compared over three walk-forward days and three seeds against five normalisation baselines and the three stages of DAIN, plus two single-stage variants that the paper does not test (scale only, gate only). Full DAIN reaches a macro-F1 of 66.3% (paper: 66.9%) against 51.4% for z-score and 58.4% for instance normalisation, and every variant lies within about one point of the published value except sample-average normalisation. The extra runs show that scaling only helps once the window has been centred: the scale alone reaches 52.0%, no better than z-score, and the gate alone does not learn at all. The analysis also shows that the shift layer acts through weight changes of order 10^{-7} on features of very different magnitude, and that the model fails mainly in quiet market periods and by missing moves rather than by predicting the wrong direction.

1 Introduction and motivation

A limit order book (LOB) lists the buy and sell orders that are waiting at each price level of a stock exchange [2]. Predicting the direction of the next mid-price move (up, stationary, or down) from the recent state of the book is a standard task in high-frequency trading: market makers use such signals to adjust their quotes, and execution algorithms use them to time their orders [3]. It is a pattern-recognition problem with three classes, where each input is a short multivariate time series of book features. In this work, each input sample (window) consists of 15 consecutive LOB snapshots, with 144 features at each snapshot.

Before such data can be given to a neural network, they must be normalised, and this is harder than it looks. Prices of different stocks lie at very different levels, the same stock drifts during the trading session and between sessions, and volumes vary by orders of magnitude. The usual remedy, z-score normalisation with the mean and standard deviation of the training data, uses fixed statistics. When the test day lies in a different price range than the training days, the normalised inputs are shifted away from the region the network has learned. This non-stationarity, together with a strong class imbalance (most short-horizon moves are "stationary") and a very low signal-to-noise ratio, are the main challenges of the task.

Deep Adaptive Input Normalization (DAIN) [1] replaces fixed normalisation by a small trainable layer that normalises every input window using its own statistics, through learned linear maps, and is trained together with the classifier. This report re-implements DAIN independently and reproduces the ablation study of the paper (Table I, MLP rows) on the FI-2010 benchmark [4]. It compares five fixed normalisation baselines with the three stages of DAIN and presents quantitative and qualitative results, along with a data-based analysis of where the method fails.

2 Background and related work

Fixed normalisation and stationary features. The simplest approach is to standardise each feature using statistics from the training set (e.g., z-score) or to rescale it to a fixed range. Both assume that the test data follow the training distribution. A second approach avoids the problem by design: the FI-2010 features [4] include, besides raw prices and volumes, hand-crafted features such as spreads, price differences and derivatives, while other work uses returns or order-flow imbalance [3] instead of price levels. These features are more stationary, but they are designed by hand and there is no guarantee that they are the best choice for a given model.

Normalisation layers. Batch Normalization [5] normalises each feature using the statistics of the current mini-batch during training and running averages at test time, so at inference it behaves like a fixed z-score. Instance Normalization [6] instead normalises each input window using its own mean and standard deviation. This removes differences in level and scale between windows, but can also remove useful information contained in those differences.

Deep LOB models. Deep networks for LOB data, such as the convolutional-recurrent DeepLOB [7] or the attention-based bilinear network TABL [8], focus on the architecture and take z-score or decimal-scaled inputs as given. DAIN addresses the step before the architecture: it is a front-end layer that can be placed before different models. This report studies it with an MLP classifier. It contains sample-wise normalisation as a special case (and z-score normalisation up to fixed statistics), and learns from the data how much of the window statistics to remove.

3 Theory and conceptual explanation

Intuition. Fixed normalisation compares each window with the training distribution, which can be problematic for a non-stationary series when the test window lies outside that distribution. Sample-wise normalisation instead compares each value with the other values in its own window. This removes the level and scale of each feature, but can also remove information carried by them. DAIN sits in between, it computes the mean and the spread of every feature over the window and passes them through learned linear maps, so the network itself decides how much of the level (baseline) and scale to remove, and which features to suppress (Fig. 1).

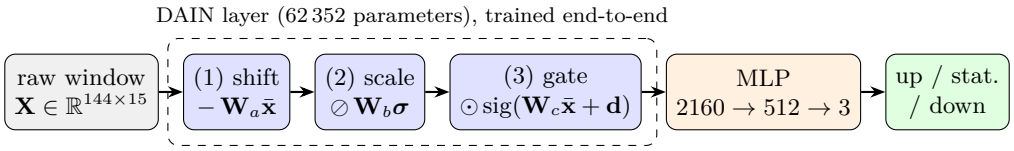


Figure 1: Pipeline. A window of the last 15 feature vectors (144 raw LOB features each) passes through the three DAIN stages, which compute their shift, scale and gate from the statistics of the window itself, and then through the MLP classifier. Bars denote means over the 15 time steps of the window.

The three stages. (1) *Adaptive shift*: subtract a learned linear combination of the window means. (2) *Adaptive scale*: divide by a learned combination of the window standard deviations (with identity matrices in stages (1) and (2), this reduces to instance normalisation). (3) *Adaptive gate*: multiply every feature by a value in $(0, 1)$ computed from the normalised window, which can switch off features that are not useful for the current window. The paper only adds the stages in this order (1, then 1+2, then 1+2+3); this report also tries stage 2 and stage 3 alone, written DAIN(2) and DAIN(3).

Design choices. The shift and scale matrices start at the identity, so training starts from a sensible normalisation. The shift is linear (no activation) so that raw inputs of size 10^5 cannot saturate it. Each stage has its own learning rate because the gradients of layers operating on raw features can have very different scales from those of the classifier.

Advantages and limitations. DAIN needs no stored statistics, adapts at test time to the current window, works with raw features and adds few parameters. Its limitations are the sensitivity to the three extra learning rates (which the authors tuned by line search), the restriction to first- and second-order window statistics, and the fact that removing the level of a window also removes information that might be predictive.

4 Mathematical formulation

Notation. An input sample is a window $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_T] \in \mathbb{R}^{d \times T}$ of T consecutive time snapshots, each represented by a feature vector $\mathbf{x}_t \in \mathbb{R}^d$ with d features. Its label $y \in \{0, 1, 2\}$ is the direction of the mid-price after the window (up, stationary, down, respectively). f_θ denotes the classifier with parameters θ. $\odot$ and $\oslash$ are element-wise product and division, and all operations on vectors are element-wise unless stated otherwise.

Stage 1, adaptive shift. With the window mean $\mathbf{a} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{x}_t$ and a learned matrix $\mathbf{W}_a \in \mathbb{R}^{d \times d}$,

$$\boldsymbol{\alpha} = \mathbf{W}_a \mathbf{a}, \qquad \tilde{\mathbf{x}}_t = \mathbf{x}_t - \boldsymbol{\alpha}. \tag{1}$$

Stage 2, adaptive scale. With the root mean square of the shifted window $\mathbf{b} = \left(\frac{1}{T}\sum_t \tilde{\mathbf{x}}_t^2 + \epsilon\right)^{1/2}$ and $\mathbf{W}_b \in \mathbb{R}^{d \times d}$,

$$\boldsymbol{\beta} = \mathbf{W}_b \mathbf{b}, \qquad \tilde{\tilde{\mathbf{x}}}_t = \tilde{\mathbf{x}}_t \oslash \boldsymbol{\beta}, \tag{2}$$

where entries $\beta_j \leq \epsilon$ are replaced by 1 to avoid division by zero.

Stage 3, adaptive gate. With the mean of the scaled window $\mathbf{c} = \frac{1}{T}\sum_t \tilde{\tilde{\mathbf{x}}}_t$, a matrix $\mathbf{W}_c$ and a bias $\mathbf{d}$,

$$\boldsymbol{\gamma} = \mathrm{sigm}(\mathbf{W}_c \mathbf{c} + \mathbf{d}) \in (0, 1)^d, \qquad \hat{\mathbf{x}}_t = \tilde{\tilde{\mathbf{x}}}_t \odot \boldsymbol{\gamma}. \tag{3}$$

Special cases. With $\mathbf{W}_a = \mathbf{I}$, DAIN performs sample-average normalisation. With $\mathbf{W}_a = \mathbf{W}_b = \mathbf{I}$ and $\boldsymbol{\gamma} = \mathbf{1}$, it reduces to instance normalisation. Replacing $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ with fixed statistics estimated from the training set gives z-score normalisation. These special cases show that DAIN can represent standard normalisation schemes and learn to move away from them during training if needed.

Single stages. Without stage 1, the scale of DAIN(2) is computed on the raw window:

$$\hat{\mathbf{x}}_t = \mathbf{x}_t \oslash \mathbf{W}_b\Big(\frac{1}{T}\sum_{t=1}^{T} \mathbf{x}_t^2 + \epsilon\Big)^{1/2}. \tag{4}$$

For features that vary little inside the window compared with their level, as prices do, this maps every value close to ± 1. DAIN(3) applies only the gate to the raw window:

$$\hat{\mathbf{x}}_t = \mathbf{x}_t \odot \mathrm{sigm}(\mathbf{W}_c \mathbf{a} + \mathbf{d}). \tag{5}$$

Since every gate value lies in $(0, 1)$, it can shrink a feature but never bring a raw price of order 10^5 to a usable scale.

Training objective. The normalised window $\hat{\mathbf{X}}$ is first flattened into a vector and then passed through a one-hidden-layer MLP:

$$f_\theta(\hat{\mathbf{X}}) = \mathbf{W}_2 \mathrm{ReLU}\left(\mathbf{W}_1 \mathrm{vec}(\hat{\mathbf{X}}) + \mathbf{b}_1\right) + \mathbf{b}_2, \tag{6}$$

where $\mathbf{W}_1$ and $\mathbf{b}_1$ are the weight matrix and bias of the hidden layer, and $\mathbf{W}_2$ and $\mathbf{b}_2$ are the weight matrix and bias of the output layer. Dropout is applied after the hidden-layer ReLU during training. The three output values are the logits for the up, stationary, and down classes. All model parameters $\Theta = \{\theta, \mathbf{W}_a, \mathbf{W}_b, \mathbf{W}_c, \mathbf{d}\}$ are trained jointly by minimising the cross-entropy loss

$$\mathcal{L}(\Theta) = \mathbb{E}_{(\mathbf{X}, y) \sim q}\left[-\log \mathrm{softmax}(f_\theta(\hat{\mathbf{X}}))_y\right], \qquad q(\mathbf{X}_i) \propto \frac{1}{n_{y_i}}, \tag{7}$$

where q is the sampling distribution and assigns each training sample a probability inversely proportional to the number n_c of samples in its class. Thus, each of the three classes contributes equally to the loss, giving

$$\mathcal{L}(\Theta) = \frac{1}{3} \sum_c \frac{1}{n_c} \sum_{i:y_i=c} \ell_i, \tag{8}$$

where ℓ_i is the cross-entropy loss for sample i. This class balancing is important because the dataset is highly imbalanced; without balancing, the classifier could be biased towards the larger class.

Gradients and per-stage learning rates. The gradient with respect to the shift matrix follows from (1):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_a} = -\sum_t \left(\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{x}}_t} \right) \mathbf{a}^\top.$$

Because $\mathbf{a}$ contains raw prices, the gradients of the DAIN stages can have very different scales from those of the classifier. Therefore, each DAIN stage $k \in \{a, b, c\}$ is assigned its own learning-rate multiplier ηm_k. Training uses RMSprop [9]:

$$\mathbf{v} \leftarrow \rho\, \mathbf{v} + (1-\rho)\, \mathbf{g}^2, \qquad \boldsymbol{\vartheta} \leftarrow \boldsymbol{\vartheta} - \eta m_k \frac{\mathbf{g}}{\sqrt{\mathbf{v}} + \epsilon}. \tag{9}$$

The division by $\sqrt{\mathbf{v}}$ normalises the gradient magnitude, while the multiplier m_k allows different learning rates for the different DAIN stages. As a result, each weight moves by roughly $\eta\, m_k$ per step. For the shift this is $10^{-4} \cdot 10^{-6} = 10^{-10}$, so over the roughly 20 000 steps of a run the shift weights can change by only about $2 \cdot 10^{-6}$: the shift matrix should stay at the identity, and stage 1 should behave like sample-average normalisation. This prediction is checked in Section 8.

Evaluation metrics. The per-class precision and recall are defined as

$$P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}, \qquad R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}.$$

The macro-F1 is

$$\text{Macro-F1} = \frac{1}{3} \sum_c \frac{2 P_c R_c}{P_c + R_c},$$

which weights the rare up and down classes as much as the frequent stationary class.
Cohen's kappa is

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is the observed accuracy and p_e is the agreement expected if the predictions were made randomly according to the observed class frequencies. Thus, $\kappa = 0$ means that the observed agreement is no better than what would be expected from such random predictions.

Complexity. DAIN adds $3d^2 + d = 62\,352$ parameters to the MLP's 1 107 971 and costs $O(dT + d^2)$ operations per window, negligible compared with the first MLP layer ($O(dT \cdot 512)$).

5 Algorithm and methodology

Input and output. Input: the last $T = 15$ LOB snapshots of one stock. Output: a probability for each of the three classes of the next mid-price move (horizon of 10 events [4]).

Evaluation protocol. FI-2010 covers 10 trading days. In the *anchored walk-forward* scheme of the paper, split k trains on days $1, \ldots, k$ and tests on day $k+1$, so the model never sees data from the future; the ablation study of the paper (Table I) uses the first three splits, and so does this report.

Training and inference. Algorithm 1 gives the procedure. The settings are those of the paper: RMSprop with base learning rate 10^{-4}, batch size 128, 20 epochs, dropout 0.5, and stage multipliers $m_a = 10^{-6}$, $m_b = 10^{-3}$, $m_c = 10$ for the three DAIN stages. Baselines use the same network and training, with the DAIN layer replaced by: nothing (raw input), z-score with the mean

Algorithm 1 Training and evaluation of DAIN + MLP for one walk-forward split.

Require: raw features of days $1..k$ (train) and day $k+1$ (test); method (DAIN stages or baseline)

build windows: for every stock, sample j uses rows $j-15, \ldots, j-1$ and the label of row j
initialise $\mathbf{W}_a, \mathbf{W}_b \leftarrow \mathbf{I}$, other weights randomly
for 20 epochs **do**
    draw N training windows with replacement, $\Pr(i) \propto 1/n_{y_i}$ (class balance)
    for each mini-batch of 128 windows **do**
        normalise with Eqs. (1) to (3); classify with the MLP
        RMSprop step (9) on (7), learning rate $10^{-4} \cdot m_k$ for DAIN stage k
    end for
end for
predict the class of every test window (dropout off); report macro-F1 and κ

and standard deviation of the training days, subtraction of the window mean (sample average), Batch Normalization on the input and Instance Normalization on the input. Each configuration is run on the three splits with three random seeds (initialisation, sampling and dropout).

6 Implementation and code snippets

The re-implementation was written from the paper in PyTorch and is collected in one notebook, `DAIN_reproduction.ipynb`, in the repository linked at the end of this report; the authors' code [10] was read to check details that the paper leaves open (window alignment, learning-rate multipliers). Instead of a Python data loader, the whole feature matrix is kept on the GPU and windows are gathered by index, which makes one 20-epoch run take about 50 seconds on average.

The DAIN layer. Lines 3 to 6 create the three stages, with the shift and scale initialised to the identity; lines 9 to 16 apply the stages listed in `stages`: $\{1\}$, $\{1,2\}$, $\{1,2,3\}$ for the variants of the paper and $\{2\}$, $\{3\}$ for the two extra ones.

```
1  class DAIN(nn.Module):
2      def __init__(self, stages, d=144, eps=1e-8):
3          super().__init__(); self.stages, self.eps = set(stages), eps
4          self.shift = nn.Linear(d, d, bias=False); nn.init.eye_(self.shift.weight)
5          self.scale = nn.Linear(d, d, bias=False); nn.init.eye_(self.scale.weight)
6          self.gate = nn.Linear(d, d)
7
8      def forward(self, x): # x: (batch, d, T)
9          if 1 in self.stages:
10             x = x - self.shift(x.mean(2)).unsqueeze(2) # (1) adaptive shift
11         if 2 in self.stages:
12             beta = self.scale(torch.sqrt((x ** 2).mean(2) + self.eps))
13             beta = torch.where(beta <= self.eps, torch.ones_like(beta), beta)
14             x = x / beta.unsqueeze(2) # (2) adaptive scale
15         if 3 in self.stages:
16             x = x * torch.sigmoid(self.gate(x.mean(2))).unsqueeze(2) # (3) adaptive gate
17         return x
```

Per-stage learning rates. Each DAIN stage is a separate parameter group of the optimiser, so that its learning rate is the base rate times its multiplier (Eq. (9)):

```
a, b, c = SUB_LR # 1e-6, 1e-3, 10
lay = net.norm.layer
groups = [{"params": classifier_params},
          {"params": lay.shift.parameters(), "lr": LR * a},
          {"params": lay.scale.parameters(), "lr": LR * b},
          {"params": lay.gate.parameters(), "lr": LR * c}]
opt = torch.optim.RMSprop(groups, lr=LR) # LR = 1e-4
```

Class-balanced training loop. Every epoch draws as many windows as there are training samples, with probability inversely proportional to the class frequency (Eq. (7)); `Xs[Wtr[b]]` gathers the windows of a batch directly on the GPU.

```
w = (1.0 / torch.bincount(ytr, minlength=3).float())[ytr] # weight 1/n_c per window
for ep in range(20):
    idx = torch.multinomial(w, len(ytr), replacement=True, generator=gen)
    for b in idx.split(128):
        loss = lossf(net(Xs[Wtr[b]]), ytr[b]) # cross-entropy
        opt.zero_grad(); loss.backward(); opt.step()
```

7 Dataset and experimental setup

Data. FI-2010 [4] contains the limit order books of five Finnish stocks traded on Nasdaq Helsinki over ten trading days in June 2010. The preprocessed file published with the authors' code [10] holds 453 975 feature vectors with 144 *raw* (not normalised) features each: prices and volumes of the ten best ask and bid levels (features 1 to 40) and derived quantities such as spreads, mid-prices, price differences, time derivatives and order intensities (features 41 to 144). Labels: 5.1% up, 89.6% stationary and 5.3% down (horizon of 10 events).

Splits. Anchored walk-forward with the first three splits, as in Table I of the paper: train on day 1 / days 1 to 2 / days 1 to 3 (46 847 / 91 546 / 124 851 windows) and test on day 2 / 3 / 4 (44 619 / 33 225 / 42 757 windows). There is no separate validation set: all hyperparameters are those of the paper and nothing was tuned here. Results are averaged over the three splits and three seeds and the error bars in the table are the standard deviations over the splits (seed-averaged), as in the paper.

Methods compared. Five baselines (no normalisation, z-score, sample average, Batch Normalization, Instance Normalization) and the three DAIN variants (stage 1, stages 1+2, stages 1+2+3), all with the same MLP, training procedure and hyperparameters (Algorithm 1). Two further variants, stage 2 alone and stage 3 alone, were added with the same settings and learning-rate multipliers; together this gives $10 \times 3 \times 3 = 90$ training runs.

Hardware and software. Laptop with an NVIDIA GeForce RTX 3050 Laptop GPU (4 GB) and 16 GB RAM; Python 3.11.9, PyTorch 2.6.0, scikit-learn 1.9.1, NumPy 2.2.6, h5py 3.16.0.

8 Quantitative results

Table 1: Reproduction of the ablation study of the paper (Table I, MLP rows), with two extra single-stage variants, DAIN (2) and DAIN (3), that are not in the paper. Macro-F1 in % (mean ± standard deviation over the three walk-forward splits) and Cohen's κ (mean). Re-implementation: three seeds per split.

	macro-F1 (%)		Cohen's κ	
normalisation	paper	re-implementation	paper	re-implementation
No normalisation	12.71 ± 13.22	12.92 ± 0.26	0.0010	0.0002
z-score	53.76 ± 0.99	51.41 ± 2.23	0.3059	0.2718
Sample average	41.80 ± 3.58	49.91 ± 6.53	0.1915	0.2512
Batch Norm.	52.72 ± 1.94	52.42 ± 1.82	0.2893	0.2856
Instance Norm.	59.13 ± 2.94	58.38 ± 3.70	0.3717	0.3597
DAIN (1)	57.37 ± 3.16	57.38 ± 4.82	0.3536	0.3576
DAIN (2)	–	52.00 ± 2.48	–	0.2877
DAIN (3)	–	9.62 ± 5.87	–	0.0000
DAIN (1+2)	66.71 ± 2.02	66.15 ± 2.40	0.4896	0.4816
DAIN (1+2+3)	66.92 ± 1.70	66.29 ± 2.31	0.4934	0.4841

The table above compares the re-implementation with the paper. The full DAIN layer reaches a macro-F1 of 66.3% ($\kappa = 0.484$), close to the published 66.9%. Most methods are within one point of the published values; the main difference is sample-average normalisation, which performs better here (49.9% vs. 41.8%) but is less stable across seeds.

Main comparisons (paired over the nine split-seed runs, standard error over splits):

- Full DAIN improves macro-F1 by 14.9 ± 0.7 points over z-score and 7.9 ± 0.8 points over Instance Normalization, showing that adapting the normalisation to each input window is more effective than using fixed training statistics or standard instance-wise normalisation.
- Considered individually, the DAIN stages behave quite differently. DAIN(1), which applies only adaptive shift, reaches a macro-F1 of 57.38%, while DAIN(2), which applies only scaling alone, reaches 52.00% ($\kappa = 0.2877$). DAIN(3), which uses only the gate, performs poorly, reaching 9.62% macro-F1 ($\kappa = 0.0000$) and assigning at least 99.96% of test windows to a single class in every run. Thus, scaling or gating alone does not achieve the performance of the shift-only variant.
- Combining shift and scale improves macro-F1 by 8.8 ± 1.6 points over shift alone, with higher performance in all 9 runs. Moreover, DAIN(1+2) outperforms DAIN(2) by 14.1 ± 0.5 points. This suggests that adaptive scaling is substantially more effective when applied after centring the input window, rather than on its own.
- Adding the gate to shift and scale improves macro-F1 by only 0.1 ± 0.2 points, with higher performance in 4 of 9 runs. This provides no clear evidence that gating offers a consistent additional benefit once shift and scale are applied.
- Without normalisation, the network reaches a macro-F1 of 12.92% ($\kappa = 0.0002$) and predicts a single class. This resembles the failure of gate-only DAIN(3), suggesting that feature gating cannot substitute for effective input normalisation. The macro-F1 of a single-class predictor depends on which class it predicts, helping explain the variability observed in these failed models.

Figure 2 puts all stage combinations side by side. Two predictions about the single stages were written down before they were run: that DAIN(2) would learn but stay clearly below DAIN(1+2), and that DAIN(3) would not learn at all. Both came true. DAIN(2) also ends up level with z-score rather than above it, which is less than expected.

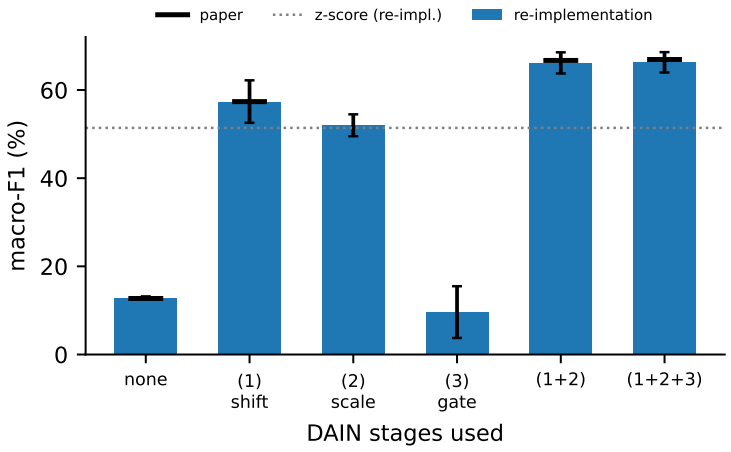


Figure 2: Macro-F1 of every combination of DAIN stages that was run (mean ± standard deviation over the splits) , with the z-score baseline as a reference line.

From Eq. (9) it was predicted before training that the shift matrix stays at the identity and that DAIN(1) would therefore equal sample-average normalisation. The first half holds: over all runs the largest change of any entry of $\mathbf{W}_a$ is 2e-7 for the full model. The second half does not: DAIN(1)

beats sample-average normalisation by $+7.5 \pm 1.0$ points (in 8 of 9 runs). Section 10 shows why these two observations are compatible.

9 Qualitative results

Non-stationarity. Figure 3 shows the problem DAIN addresses. A z-score fitted on the training days maps each stock to a fixed range, but the test day of a drifting stock falls partly outside it; a window-based normalisation does not have this problem because it only looks at the last 15 vectors.

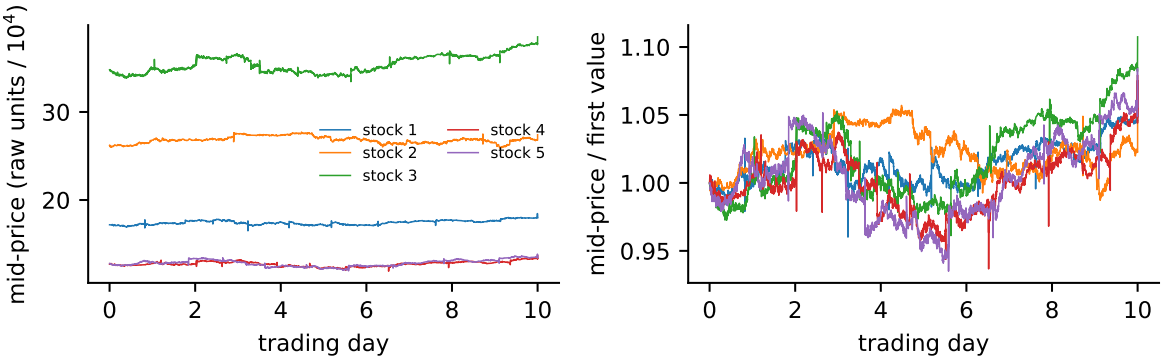


Figure 3: Left: mid-price of the five stocks over ten trading days in raw units. Right: the same prices relative to their first observed value.

Which mistakes the models make. The confusion matrices (Fig. 4) show where the gain of DAIN comes from. With DAIN, 54.1% of the true up and down moves are detected with the correct direction (z-score: 44.6%) and 45.3% are predicted as stationary (z-score: 50.2%). The errors in the other two directions change even more: the z-score model predicts the *opposite* direction for 5.2% of the moves and recognises only 84.6% of the stationary windows, whereas with DAIN direction errors almost disappear (0.6%) and 93.6% of the stationary windows are recognised. DAIN therefore detects more moves and, above all, raises far fewer false and wrong-direction signals.

Figure 4 shows how the DAIN stages affect move detection and stationary-class recognition. DAIN(1) correctly identifies the direction of more moves than the other variants (63.8%) and rarely predicts the opposite direction (1.0%). However, it correctly recognises only 82.6% of stationary windows, leading to many false alarms. Adding the scale raises stationary recognition to 92.8% while keeping direction errors low (0.7%), although fewer moves are correctly classified (56.2%). The confusion matrix of DAIN(1+2) is close to that of full DAIN, consistent with the gate providing little additional benefit.

DAIN(2) makes a different trade-off: among the models that learn, it labels the largest share of actual moves as non-stationary, with only 34.2% classified as stationary. However, it predicts the wrong direction for 7.7% of moves and correctly recognises only 78.3% of stationary windows. These errors offset its greater move detection, leaving its macro-F1 close to that of z-score.

DAIN(3) behaves very differently: its three confusion-matrix rows are nearly identical. In every run, it predicts a single class for all test windows (3 runs predict "up", 2 predict "stationary", and 4 predict "down"). The similar rows arise because the matrices aggregate runs with different constant predictions. Thus, its apparent directional predictions do not reflect meaningful discrimination between input windows.

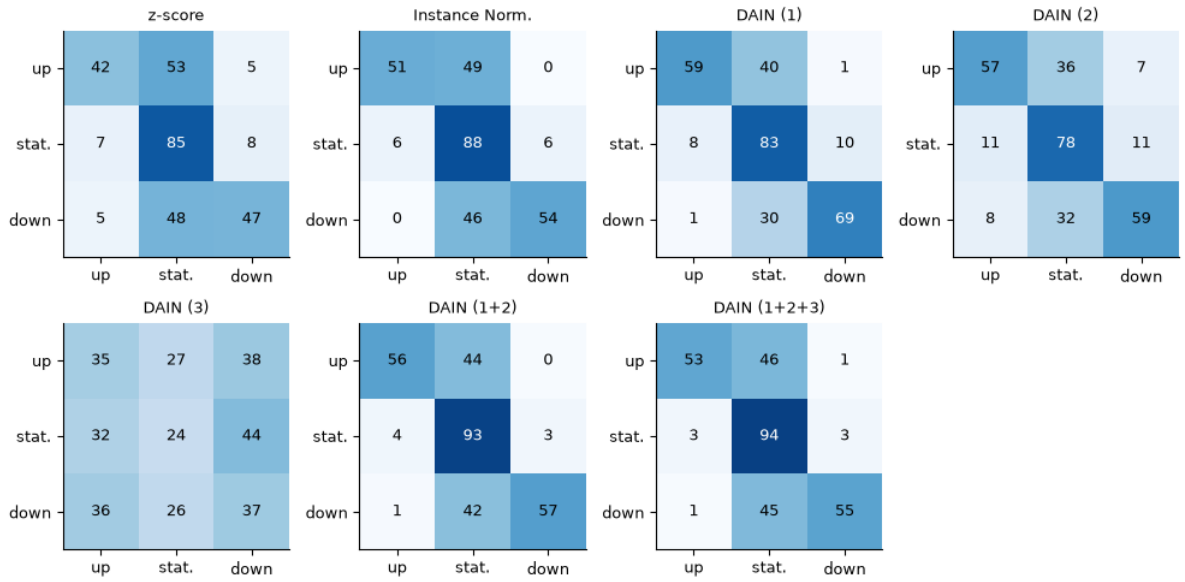


Figure 4: Confusion matrices of z-score, instance normalisation and all DAIN variants (rows: true class, in % of the row), summed over the three splits and three seeds

Predictions over time. Figure 5 shows the predictions along one test day. The model fires mostly around the jumps of the mid-price and stays silent in flat stretches, which is the desired behaviour. Most clusters of true moves are caught with the right direction, usually only on some of their windows, which is the "missed move" error of Fig. 4. The visible false signals are isolated windows in flat stretches (for example around samples 1100 and 1220), where the model predicts a move that does not happen.

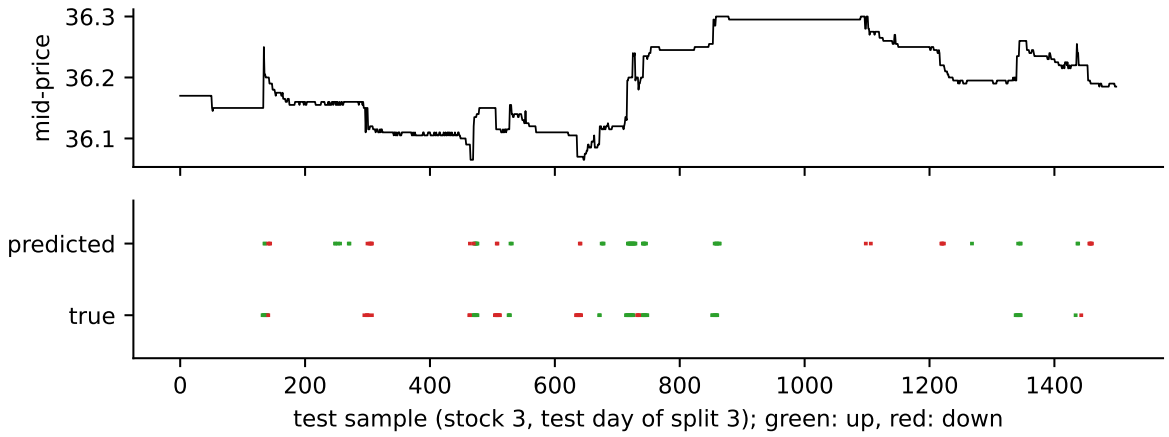


Figure 5: A part of one test day (stock 3, test day of the third split) with the full DAIN model: mid-price (top) and the true and predicted up/down labels (bottom; stationary windows are left blank).

What DAIN learned. Figure 6 illustrates the third stage of the layer. Volume features have gate values close to one (mean 0.97), whereas the price-level features are more strongly suppressed (mean 0.35), except for the best quotes; the derived features lie in between (mean 0.74). After shifting and scaling, the price levels of the ten levels carry almost the same information (they move together), so the gate keeps the volumes, which describe supply and demand at each level, and damps the redundant prices. This pattern is consistent with the role of order-book imbalance in short-term price prediction [3]. However, the gate does not noticeably improve macro-F1 (Section 8), suggesting that its feature-wise reweighting adds little benefit beyond the shift and scale stages.

The downstream MLP may already learn to emphasise or suppress features through its first-layer weights, which could explain why adding the gate makes little difference to the overall performance.

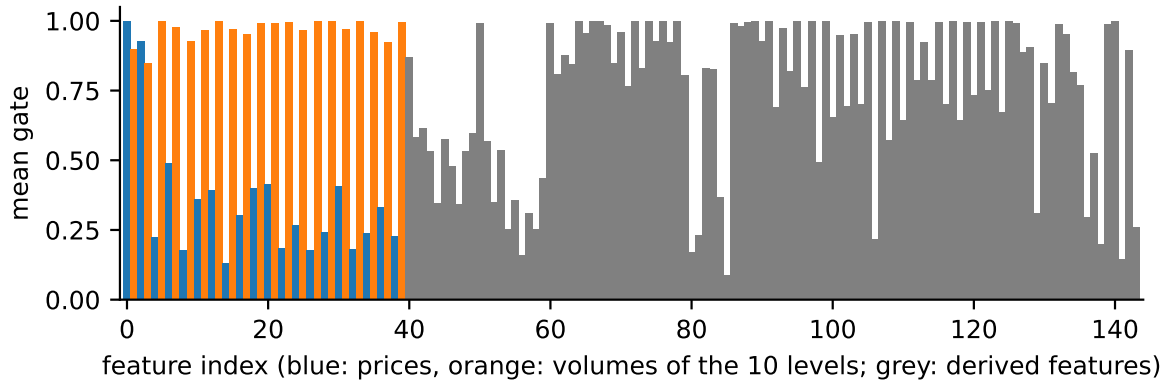


Figure 6: Mean gate value $\bar{\gamma}_j$ for each of the 144 features across the test windows, for the full DAIN model on the third split with seed 0.

10 Analysis and discussion

Why DAIN works. The gain comes from centring and scaling together. Price changes of a stock with a high price, or in a volatile period, are larger in absolute terms than those of a cheap stock in a calm period; after subtracting the window mean and dividing by the window spread, a large move means the same thing for every stock and every hour. Instance normalisation does this as well, but with a fixed identity matrix, whereas DAIN learns a combination of the window statistics and gains a further +7.9 points. The z-score baseline cannot adapt at all and is the weakest of the working methods.

What each stage does to the predictions. The confusion matrices (Fig. 4) give each stage a clear role. Centring the window (the shift) is what gets the direction right: every variant that centres, DAIN(1), DAIN(1+2) and full DAIN, makes at most 1.0% direction errors, while the scale-only DAIN(2) makes 7.7%. Dividing by the window spread (the scale) is what makes the model selective: with it, the share of stationary windows recognised rises from 82.6% for DAIN(1) to 92.8% for DAIN(1+2), at the cost of some caught moves (63.8% to 56.2%). Because most windows are stationary, removing these false alarms is worth more in macro-F1 than the lost hits, which is where the +8.8 points of the scale come from. The gate changes neither of the two.

Why each stage fails on its own. The single-stage runs show what goes wrong when one half is missing. Without the shift, DAIN(2) divides each raw feature by its root mean square, which for a price is almost the price itself. The level differences between stocks disappear, but every value now sits close to ± 1, and the small changes inside the window that carry the signal remain tiny deviations from that constant. A division can never remove this constant, whatever the weights are (and they move by at most 6e-4 during training). The MLP then sees inputs that resemble relative prices: enough to learn something, as much as with z-score, but it reacts to small noisy deviations, which matches the many false and wrong-direction signals of Fig. 4. Centring first removes the constant, and dividing afterwards by the spread around the mean blows the changes up to a usable size; this is why putting stage 1 in front of stage 2 adds +14.1 points. Without the other two stages, the gate cannot help at all: a factor between 0 and 1 cannot turn raw prices of order 10^5 into inputs a network can train on, so DAIN(3) collapses to a single class like the network without normalisation. The gate is only useful as a final reweighting of features that are already normalised, and even there its effect is too small to measure.

Why the shift layer helps although it does not move. The weights of the shift matrix change by at most 2e-7, as predicted, yet DAIN(1) is clearly better than sample-average normalisation. The reason is the scale of the raw features: prices are of order 10^5 and some volume-based

features exceed 10^6, while eleven features have an average magnitude below 10^{-2}. A change of 10^{-7} in an off-diagonal weight that multiplies a price adds a shift of about 10^{-2} to another feature, which is enormous for a feature whose values are tiny. Measured on the test windows of the inspected model (largest weight change 2e-7), the shift induced by $\mathbf{W}_a - \mathbf{I}$ exceeds the within-window standard deviation for 15 of the 144 features. The shift layer therefore does learn, by moving the small features, and "the weights stay at the identity" does not mean "the function stays at the identity". This also explains why the authors needed such an extreme multiplier (10^{-6}): any larger step would destroy the small features.

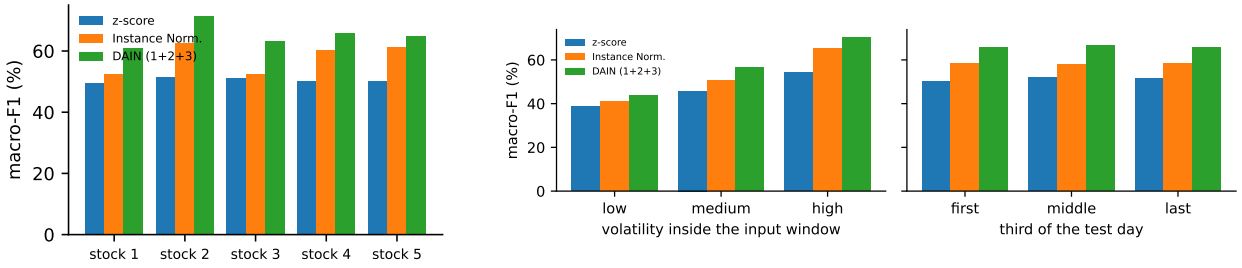


Figure 7: Failure analysis (macro-F1, mean over splits and seeds). Left: per stock. Middle: test windows grouped by the volatility of the mid-price inside the window (terciles). Right: windows grouped by the third of the trading day.

Failure cases, measured. Three analyses of the saved test predictions locate where the models fail (Fig. 7):

1. *Quiet markets.* The strongest effect is volatility. For full DAIN the macro-F1 is 70.4% in the most volatile third of the windows but only 43.7% in the quietest third (z-score: 54.4% and 38.6%). In a quiet window the mid-price hardly moves, the next move is driven by orders that have not arrived yet, and the stationary threshold separates moves of almost the same size. Normalising by the window spread cannot help here: dividing a tiny spread by itself amplifies noise.

2. *Some stocks are harder.* Full DAIN achieves a macro-F1 of 71.5% on stock 2, compared with 61.1% on stock 1. Although DAIN performs better than z-score normalisation on every stock, its performance still varies considerably across stocks. This suggests that some stocks remain more difficult to predict, even with adaptive normalisation.

3. *Missed moves.* 45.3% of the up and down moves are predicted as stationary (Fig. 4); the model fails mainly by staying silent, not by predicting the wrong direction (0.6%). For a trading application this is the less costly kind of error.

The time of day, by contrast, has almost no effect (DAIN between 65.7% and 66.5% in the three thirds of the day), so the failures are not caused by the opening or closing period of the market.

Hyperparameters and sensitivity. The learning-rate multipliers are the most delicate hyperparameters: the shift multiplier is so small that the shift weights move only by 10^{-7}, and the analysis above shows that even this is enough to change the small features. The seed variation is a further indicator of stability: 0.4 points for full DAIN against 1.2 for z-score, 2.8 for the scale-only variant and 3.9 for sample average.

Compute. One 20-epoch run takes about 50 seconds on the laptop GPU, and all 90 runs of this report took a little over an hour. DAIN adds 62 352 parameters (6% of the MLP) and a negligible cost per window.

Limitations. Only the MLP and the first three walk-forward splits of the paper's ablation study were reproduced; the CNN and GRU results, the evaluation across all days, and the second dataset were left out. The prediction horizon is fixed at 10 events, and macro-F1 does not establish whether the predictions translate into profitable trades after transaction costs. Finally, the results are based on ten days of data for five stocks from 2010, so they may not generalise to other periods

or markets.

11 Conclusion

DAIN was re-implemented from the paper, and its ablation study was reproduced on the FI-2010 dataset using the authors' data. The full DAIN layer achieves a macro-F1 of 66.3%, compared with 66.9% in the paper. Testing the DAIN stages separately showed that the main improvement comes from combining shift and scale. Scaling alone performs similarly to z-score normalisation (52.0%), but adding shift before scaling improves macro-F1 by +14.1 points. And the gate alone does not learn at all and adds nothing measurable on top of the other two.

The confusion matrices help explain these results. The shift stage helps the model identify the correct direction of price moves, while adding scale improves the recognition of stationary periods and reduces false alarms. The shift stage works through very small changes in its weights, of the order of 10^{-7}, even though the input features have very different magnitudes. The failure analysis also showed that the model performs worse during quiet market periods and often misses actual price moves by predicting them as stationary, rather than predicting the opposite direction.

Future work could test other network architectures and prediction horizons, extend the evaluation to all walk-forward days, and use a better-conditioned shift stage, such as one based on standardised statistics, to avoid the need for an extreme learning-rate multiplier. Finally, the model should be evaluated in terms of trading profitability after transaction costs, since classification performance alone does not show whether its predictions can lead to profitable trades.

Use of external resources

Data: FI-2010 [4], in the preprocessed form published with the authors' code [10]. **Code:** the authors' repository [10] was read to check the window alignment and the learning-rate multipliers; it was not run. **Implemented independently:** data windowing, the DAIN layer, all baseline normalisations, the MLP, the class-balanced training loop, the evaluation, the failure analyses and all figures and tables. All experiments were designed and run for this report. **Code of this report:** the complete implementation, including a notebook that reproduces all experiments, results and figures, is available at GitHub Repository.

References

[1] Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3760–3765, 2020.

[2] Martin D. Gould, Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison. Limit order books. *Quantitative Finance*, 13(11):1709–1742, 2013.

[3] Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014.

[4] Adamantios Ntakaris, Martin Magris, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[6] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.

[7] Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019.

[8] Dat Thanh Tran, Alexandros Iosifidis, Juho Kanniainen, and Moncef Gabbouj. Temporal attention-augmented bilinear network for financial time-series data analysis. *IEEE Transactions on Neural Networks and Learning Systems*, 30(5):1407–1418, 2019.

[9] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5: RMSProp, divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.

[10] Nikolaos Passalis. DAIN: Deep adaptive input normalization for time series forecasting (PyTorch implementation and preprocessed FI-2010 data). https://github.com/passalis/dain, 2019. Accessed October 2026.